

[보고서] 객체지향 디자인 패턴의 기본기: SOLID 원칙 분석

이번 영상을 통해 객체지향 프로그래밍을 할 때 단순히 문법을 아는 것뿐만 아니라, 유지보수하기 좋고 유연한 구조를 설계하는 기준이 왜 필요한지 깊이 있게 이해할 수 있었다. 선배 개발자들이 시행착오를 겪으며 만들어 낸 5가지 설계 지침인 SOLID 원칙이 각각 어떤 문제를 해결하고 어떤 방향을 제시하는지 직접 코드를 비교하며 정리해보았다.

1.SOLID 원칙 세부 분석

1.1. SRP (단일 책임 원칙)

처음에 코드를 짤 때 가장 쉽게 저지르는 실수가 하나의 클래스에 너무 많은 기능을 때려 넣는 것인데, SRP는 모든 클래스가 단 하나의 책임만 가져야 한다는 점을 명확히 짚어준다. 예를 들어 UserService 클래스 하나에서 DB 저장, 이메일 발송, 로깅을 다 처리하면 코드가 복잡해질 뿐만 아니라 수정할 이유가 여러 개가 생겨서 결국 다른 기능까지 건드리게 되는 부작용이 생긴다. 이를 각각의 전담 클래스로 쪼개야 클래스의 목적이 직관적으로 보이고, 나중에 리팩토링이나 재사용을 할 때 훨씬 수월하다는 것을 배웠다.

1.2. OCP (개방-폐쇄 원칙)

```
/out:ocp2.exe
ocp2.obj
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2> .\ocp2.exe
Generating PDF report...
Generating HTML report...
Generating XML report...
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2>
```

Ocp2번 예제

소프트웨어는 확에는 열려 있고 변경에는 닫혀 있어야 한다는 말이 처음에는 와닿지 않았지만, 레포트 생성 예제를 보니 이해가 되었다. PDF나 HTML을 처리하는 코드 안에 조건문으로 로직을 묶어두면 나중에 XML 같은 새로운 형식이 추가될 때마다 기존 메서드를 직접 뜯어고쳐야 하는 위험이 생긴다. 결국 공통 인터페이스를 두고 새로운 형식이 필요할 때 클래스만 새로 만들어 확장하는 구조를 취해야, 기존 코드를 건드리면서 발생할 수 있는 런타임 오류나 부작용을 원천적으로 차단할 수 있다는 점이 인상 깊었다.

1.3. LSP (리스코프 치환 원칙)

```
Copyright (C) Microsoft Corporation. All rights reserved.

/out:lsp1.exe
lsp1.obj
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2> .\lsp1.exe
Bird is flying
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2>
```

LSP1번예제

```
/out:lsp2.exe
lsp2.obj
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2> .\lsp2.exe
Bird is eating
sparrow is flying
Bird is eating
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2>
```

LSP2번예제

자식 클래스는 언제나 부모 클래스를 대체할 수 있어야 한다는 원칙은 상속을 설계할 때 가장 경계해야 할 부분을 짚어준다. 단순히 문법적으로 상속이 된다고 해서 아무 클래스나 묶어버리면, 부모가 기대하던 동작을 자식이 제대로 수행하지 못해 결국 예외나 에러를 유발하게 된다. 다형성을 안전하게 쓰기 위해서는 부모의 규약을 자식이 온전하게 지킬 수 있는 상속 관계인지 항상 점검해야 한다는 것을 깨달았다.

1.4. ISP (Interface Segregation Principle: 인터페이스 분리 원칙)

```
/out:isp1.exe
isp1.obj
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2> .\isp1.exe
Employee is working
Employee is eating
Robot is working
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2>
```

isp1번예제

```
/out:isp2.exe
isp2.obj
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2> .\isp2.exe
Employee is working
Employee is eating
Robot is working
PS C:\Users\lkh06\Desktop\Git\Robit_intelligence_cpp\src\day_3\hw2>
```

isp2번예제

범용적인 인터페이스 하나를 크게 만드는 것보다 클라이언트에 맞춰 작게 쪼개는 것이 중요하다는 원칙이다. 공장 노동자 예시에서 사람과 달리 식사를 하지 않는 로봇 직원에게 공통 인터페이스를 강제하면 잉여 메서드가 생기는 문제가 발생하는데, 인터페이스를 작업용과 식사 용도로 나누어 필요한 것만 골라 구현하게 만드는 방식을 보면서 인터페이스 설계의 세심함이 왜 필요한지 느낄 수 있었다.

1.5. DIP (의존성 역전 원칙)

고수준 모듈이 저수준 모듈의 구체적인 구현에 의존하면 안 되고, 둘 다 추상화에 의존해야 한다는 개념이다. 스위치와 선풍기 예시에서 스위치가 특정 선풍기 클래스에 직접 종속되어 있으면 선풍기 내부 코드가 바뀔 때 스위치까지 다 뜯어고쳐야 하고 다른 기기도 제어할 수 없는 한계에 부딪힌다. 중간에 인터페이스를 두어 의존성을 역전시키면 스위치는 선풍기뿐만 아니라 그 인터페이스를 구현한 어떤 전자기기도 제어할 수 있는 유연한 구조가 된다는 점이 매우 설득력 있게 다가왔다.

2.추가 학습:

결합도와 응집도 SOLID 원칙들을 공부하면서 이 원칙들이 궁극적으로 지향하는 핵심 판단 기준인 '결합도(Coupling)'와 '응집도(Cohesion)'의 개념에 대해서도 추가로 알아보았다. 결합도는 모듈(클래스) 간의 의존 정도를 나타내며, 코드를 수정할 때 다른 곳까지 연쇄적으로 영향을 미치지 않도록 낮게 유지해야 한다. DIP와 OCP 같은 원칙들이 바로 이 결합도를 낮추기 위한 대표적인 방법임을 이해하게 되었다. 응집도는 하나의 모듈 안에 연관된 기능들이 얼마나 밀접하게 모여 있는지를 나타내며, SRP를 통해 클래스의 책임을 하나로 좁힐수록 응집도가 높아져 코드가 명확해진다. 결국 좋은 설계란 결합도는 낮추고 응집도는 높이는 방향이라는 점을 깨닫고 나니, SOLID 원칙들이 왜 필수적인지 더 확실히 체감할 수 있었다.

3.결론 및 시사점

이번에 공부한 SOLID 원칙들은 결국 코드를 짤 때 결합도를 낮추고 응집도를 높여 변화에 유연하게 대처할 수 있도록 만드는 공통된 철학을 담고 있다. 각각의 원칙이 서로 유기적으로 연결되어 있으며, 앞으로 다양한 객체지향 디자인 패턴을 공부할 때 뼈대가 되는 핵심 기준이 되므로 실제 코드를 작성할 때 이 원칙들을 염두에 두고 설계하는 습관을 들여야겠다.